

UNIVERSIDAD
NACIONAL
DE COLOMBIA

SEDE MEDELLIN
FACULTAD DE CIENCIAS
DEPARTAMENTO DE ESTADISTICA

ACTIVIDAD 2 DE PROGRAMACION ORIENTADA A OBJETOS

Presentado por:
Maria Paula Barrera Avella - 1001220174

Presentado a:
Walter Hugo Arboleda Mazo

Asignatura:
Programación orientada a objetos

Medellin, Antioquia
8 de octubre
2026

2.1 clase persona

ejercicio 2.1 clase persona

class Persona:

Construcción de los datos

```
def __init__(self, nombre, apellidos, numero_documento_identidad, ano_nacimiento):  
    self.nombre = nombre  
    self.apellidos = apellidos  
    self.numero_documento_identidad = numero_documento_identidad  
    self.ano_nacimiento = ano_nacimiento
```

impresión de los datos

```
def imprimir(self):  
    print("Nombre =", self.nombre)  
    print("Apellidos =", self.apellidos)  
    print("Número de documento de identidad =", self.numero_documento_identidad)  
    print("Año de nacimiento =", self.ano_nacimiento)  
    print()
```

Programa

p1 = Persona("Pedro", "Pérez", "1053121010", 1998)

p2 = Persona("Luis", "León", "1053223344", 2001)

p1.imprimir()

p2.imprimir()

Persona
Atributos - nombre: string - apellidos: string - numero_documento_identidad: string - ano_nacimiento: int
Métodos + __init__(nombre: string, apellidos: string, numero_documento_identidad: string, ano_nacimiento: int) + imprimir(): void

2.2 clase planeta

#Ejercicio 2.2 clase Planeta

from enum import Enum

class TipoPlaneta(Enum):

 GASEOSO = 1

 TERRESTRE = 2

 ENANO = 3

class Planeta:

Construccion de los datos

 def __init__(

 self,

 nombre,

 cantidad_satelites,

 masa,

 volumen,

 diametro,

 distancia_sol,

 tipo,

 es_observable

):

 self.nombre = nombre

 self.cantidad_satelites = cantidad_satelites

 self.masa = masa

 self.volumen = volumen

 self.diametro = diámetro

 self.distancia_sol = distancia_sol

 self.tipo = tipo

 self.es_observable = es_observable

Imprimir los datos

 def imprimir(self):

 print("Nombre del planeta =", self.nombre)

 print("Cantidad de satélites =", self.cantidad_satelites)

 print("Masa del planeta =", self.masa)

 print("Volumen del planeta =", self.volumen)

 print("Diámetro del planeta =", self.diametro)

 print("Distancia al Sol =", self.distancia_sol)

 print("Tipo de planeta =", self.tipo.name)

 print("Es observable =", self.es_observable)

Calcular la densidad

 def calcular_densidad(self):

 return self.masa / self.volumen

Determinar si es un planeta exterior

 def es_planeta_exterior(self):

 UA = 149597870

 limite = UA * 3.4

 if self.distancia_sol > limite:

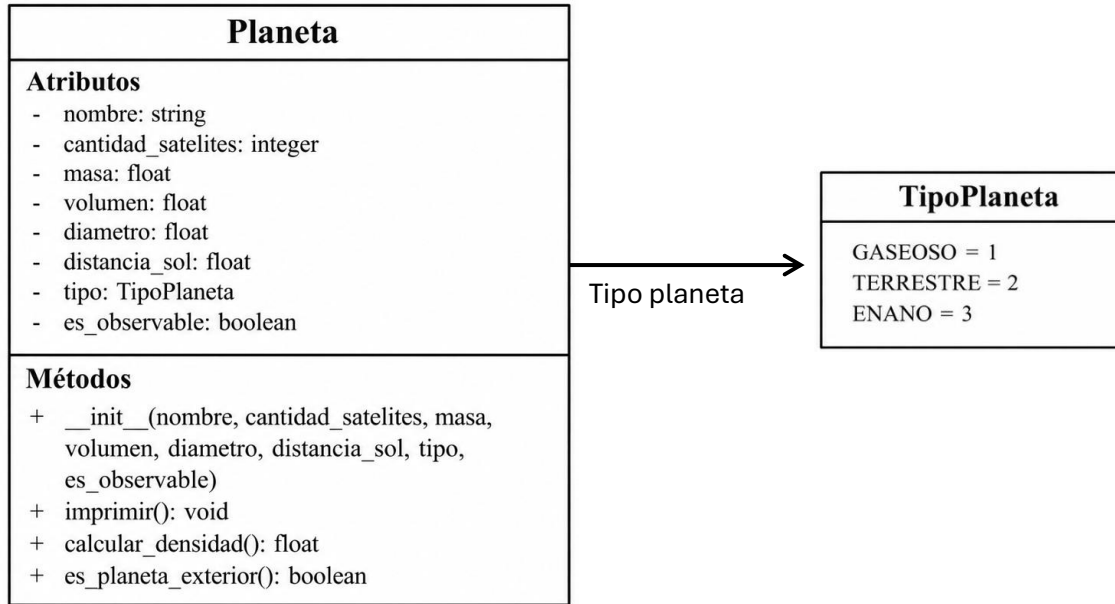
 return True

```

        else:
            return False
# Programa principal
p1 = Planeta(
    nombre="Tierra",
    cantidad_satelites=1,
    masa=5.9736E24,
    volumen=1.08321E12,
    diametro=12742,
    distancia_sol=150000000,
    tipo=TipoPlaneta.TERRESTRE,
    es_observable=True
)
p1.imprimir()
print("Densidad del planeta =", p1.calcular_densidad())
print("Es planeta exterior =", p1.es_planeta_exterior())
print()
p2 = Planeta(
    nombre="Júpiter",
    cantidad_satelites=79,
    masa=1.899E27,

    volumen=1.4313E15,
    diametro=139820,
    distancia_sol=750000000,
    tipo=TipoPlaneta.GASEOSO,
    es_observable=True
)
p2.imprimir()
print("Densidad del planeta =", p2.calcular_densidad())
print("Es planeta exterior =", p2.es_planeta_exterior())

```



2.3 Automovil

#Ejercicio 2.3 Automóvil

```
from enum import Enum
```

```
class TipoCombustible(Enum):
```

```
    GASOLINA = 1
```

```
    BIOETANOL = 2
```

```
    DIESEL = 3
```

```
    BIODIESEL = 4
```

```
    GAS_NATURAL = 5
```

```
class TipoAutomovil(Enum):
```

```
    CIUDAD = 1
```

```
    SUBCOMPACTO = 2
```

```
    COMPACTO = 3
```

```
    FAMILIAR = 4
```

```
    EJECUTIVO = 5
```

```
    SUV = 6
```

```
class TipoColor(Enum):
```

```
    BLANCO = 1
```

```
    NEGRO = 2
```

```
    ROJO = 3
```

```
    NARANJA = 4
```

```
    AMARILLO = 5
```

```
    VERDE = 6
```

```
    AZUL = 7
```

```
    VIOLETA = 8
```

```
class Automovil:
```

```

def __init__(self, marca, modelo, motor, tipo_combustible,
             tipo_automovil, numero_puertas, cantidad_asientos,
             velocidad_maxima, color):
    self.marca = marca
    self.modelo = modelo
    self.motor = motor
    self.tipo_combustible = tipo_combustible
    self.tipo_automovil = tipo_automovil
    self.numero_puertas = numero_puertas
    self.cantidad_asientos = cantidad_asientos
    self.velocidad_maxima = velocidad_maxima
    self.color = color
    self.velocidad_actual = 0

# Get
def get_marca(self):
    return self.marca
def get_modelo(self):
    return self.modelo
def get_motor(self):
    return self.motor
def get_tipo_combustible(self):
    return self.tipo_combustible
def get_tipo_automovil(self):
    return self.tipo_automovil
def get_numero_puertas(self):
    return self.numero_puertas
def get_cantidad_asientos(self):
    return self.cantidad_asientos
def get_velocidad_maxima(self):
    return self.velocidad_maxima
def get_color(self):
    return self.color
def get_velocidad_actual(self):
    return self.velocidad_actual

# Set
def set_marca(self, marca):
    self.marca = marca
def set_modelo(self, modelo):
    self.modelo = modelo
def set_motor(self, motor):
    self.motor = motor
def set_tipo_combustible(self, tipo_combustible):
    self.tipo_combustible = tipo_combustible
def set_tipo_automovil(self, tipo_automovil):
    self.tipo_automovil = tipo_automovil
def set_numero_puertas(self, numero_puertas):
    self.numero_puertas = numero_puertas

```

```

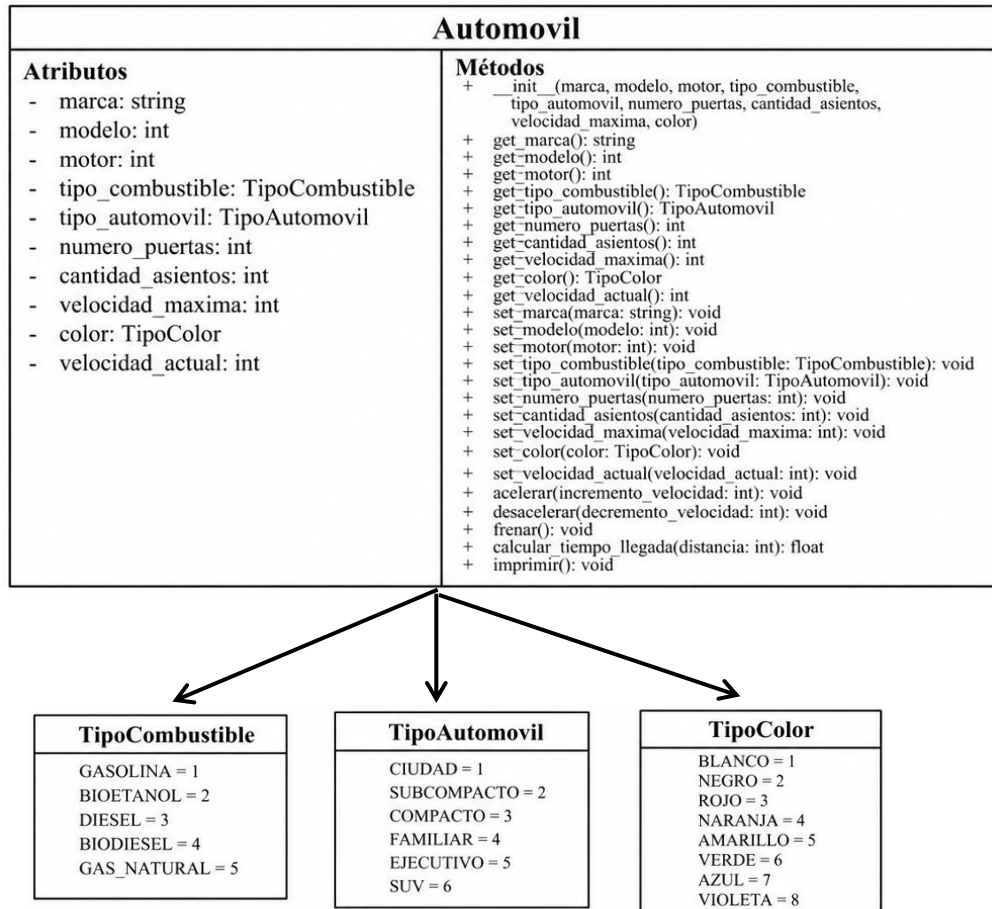
def set_cantidad_asientos(self, cantidad_asientos):
    self.cantidad_asientos = cantidad_asientos
def set_velocidad_maxima(self, velocidad_maxima):
    self.velocidad_maxima = velocidad_maxima
def set_color(self, color):
    self.color = color
def set_velocidad_actual(self, velocidad_actual):
    self.velocidad_actual = velocidad_actual
def acelerar(self, incremento_velocidad):
    if self.velocidad_actual + incremento_velocidad <= self.velocidad_maxima:
        self.velocidad_actual += incremento_velocidad
    else:
        print("No se puede superar la velocidad máxima.")
def desacelerar(self, decremento_velocidad):
    if 0 <= self.velocidad_actual - decremento_velocidad:
        self.velocidad_actual -= decremento_velocidad
    else:
        print("No se puede decrementar a una velocidad negativa.")
def frenar(self):
    self.velocidad_actual = 0
def calcular_tiempo_llegada(self, distancia):
    if self.velocidad_actual == 0:
        print("No se puede calcular el tiempo con velocidad cero.")
        return None
    return distancia / self.velocidad_actual
def imprimir(self):
    print("Marca:", self.marca)
    print("Modelo:", self.modelo)
    print("Motor:", self.motor)
    print("Tipo de combustible:", self.tipo_combustible.name)
    print("Tipo de automóvil:", self.tipo_automovil.name)
    print("Número de puertas:", self.numero_puertas)
    print("Cantidad de asientos:", self.cantidad_asientos)
    print("Velocidad máxima:", self.velocidad_maxima)
    print("Color:", self.color.name)
# Programa principal
auto1 = Automovil(
    "Ford", 2018, 3, TipoCombustible.DIESEL,
    TipoAutomovil.EJECUTIVO, 5, 6, 250, TipoColor.NEGRO
)
auto1.imprimir()
auto1.set_velocidad_actual(100)
print("Velocidad actual:", auto1.get_velocidad_actual())
auto1.acelerar(20)
print("Velocidad actual:", auto1.get_velocidad_actual())
auto1.desacelerar(50)
print("Velocidad actual:", auto1.get_velocidad_actual())

```

```

auto1.frenar()
print("Velocidad actual:", auto1.get_velocidad_actual())
auto1.desacelerar(20)

```



2.4 Figuras geométricas

#Ejercicio 2.4 Figuras geométricas

```
import math
```

```
class Circulo:
```

```
    def __init__(self, radio):
        self.radio = radio
```

```
    def calcular_area(self):
```

```
        return math.pi * self.radio ** 2
```

```
    def calcular_perimetro(self):
```

```
        return 2 * math.pi * self.radio
```

```
class Rectangulo:
```

```
    def __init__(self, base, altura):
```

```

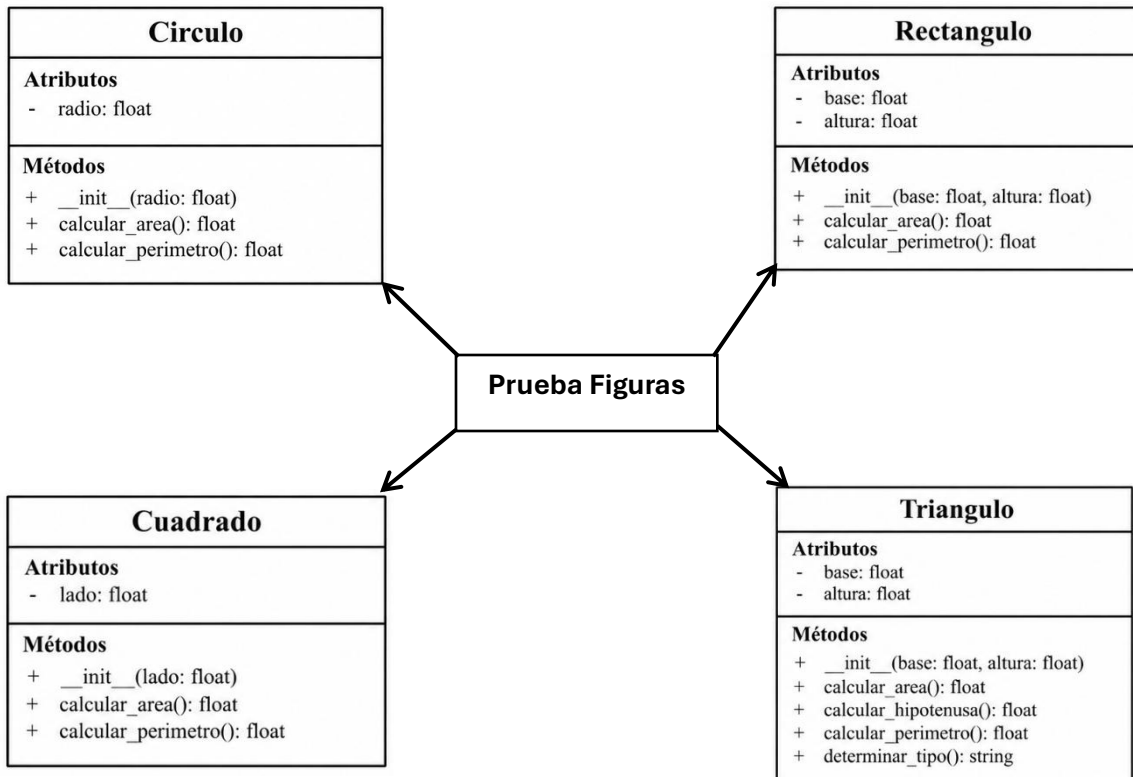
        self.base = base
        self.altura = altura
    def calcular_area(self):
        return self.base * self.altura
    def calcular_perimetro(self):
        return 2 * self.base + 2 * self.altura
class Cuadrado:
    def __init__(self, lado):
        self.lado = lado
    def calcular_area(self):
        return self.lado ** 2
    def calcular_perimetro(self):
        return 4 * self.lado
class Triangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura
    def calcular_area(self):
        return (self.base * self.altura) / 2
    def calcular_hipotenusa(self):
        return math.sqrt(self.base ** 2 + self.altura ** 2)
    def calcular_perimetro(self):
        return self.base + self.altura + self.calcular_hipotenusa()
    def determinar_tipo(self):
        hipotenusa = self.calcular_hipotenusa()
        if self.base == self.altura == hipotenusa:
            return "Equilátero"
        elif (self.base == self.altura or
              self.base == hipotenusa or
              self.altura == hipotenusa):
            return "Isósceles"
        else:
            return "Escaleno"
# Programa principal
figura1 = Circulo(2)
figura2 = Rectangulo(1, 2)
figura3 = Cuadrado(3)
figura4 = Triangulo(base=3, altura=5)
print("CÍRCULO")
print("Área =", figura1.calcular_area())
print("Perímetro =", figura1.calcular_perimetro())
print("\nRECTÁNGULO")
print("Área =", figura2.calcular_area())
print("Perímetro =", figura2.calcular_perimetro())
print("\nCUADRADO")
print("Área =", figura3.calcular_area())
print("Perímetro =", figura3.calcular_perimetro())

```

```

print("\nTRIÁNGULO")
print("Área =", figura4.calcular_area())
print("Hipotenusa =", figura4.calcular_hipotenusa())
print("Perímetro =", figura4.calcular_perimetro())
print("Es un triángulo", figura4.determinar_tipo())

```



2.5

#Ejercicio 2.5: Cuenta bancaria

```

from enum import Enum
class TipoCuenta(Enum):
    AHORROS = 1
    CORRIENTE = 2
class CuentaBancaria:
    def __init__(self, nombres_titular, apellidos_titular,
                 numero_cuenta, tipo_cuenta):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0
    def imprimir(self):

```

```

    print("Nombres del titular:", self.nombres_titular)
    print("Apellidos del titular:", self.apellidos_titular)
    print("Número de cuenta:", self.numero_cuenta)
    print("Tipo de cuenta:", self.tipo_cuenta.name)
    print("Saldo:", self.saldo)
def consultar_saldo(self):
    print("El saldo actual es:", self.saldo)
def consignar(self, valor):
    if valor > 0:
        self.saldo += valor
        print(
            "Se ha consignado $", valor,
            "en la cuenta. El nuevo saldo es $", self.saldo
        )
        return True
    else:
        print("El valor a consignar debe ser mayor que cero.")
        return False
def retirar(self, valor):
    if valor > 0 and valor <= self.saldo:
        self.saldo -= valor
        print(
            "Se ha retirado $", valor,
            "de la cuenta. El nuevo saldo es $", self.saldo
        )
        return True
    else:
        print("El valor a retirar debe ser mayor que cero "
            "y no superar el saldo actual.")
        return False
# Programa principal
cuenta = CuentaBancaria(
    "Pedro", "Pérez", 123456789, TipoCuenta.AHORROS
)
cuenta.imprimir()
cuenta.consignar(200000)
cuenta.consignar(300000)
cuenta.retirar(400000)
cuenta.consultar_saldo()

```

